Unit 2: Data Types and Control Structures

lab Exercise

2.1 Variables and Binding

- Static vs. dynamic typing
- Type inference (eg. Haskell, Type Script
- Memory management (stack, heap, garbage collection.

1. Variable and Binding

Goal: Understand how variable store values and how rebinding works.

Language: Python

Exercise: Basic Variable Binding

Fill in the blanks!

* Create a variable and bind it to a value

```
n = 5
print ("value of n is: ", n)
```

* Re-bind n to a new value

```
n = 4
Print ("Now n is: ", n)
```

Questions to answers:

1. what was the first value of n? → 5
2. what value did you re-bind n to.? → 4
3. Does re-binding change the old value or just point the variable name somehere else.

→ write your answer:- yes, rebind change the old value.

2. Static vs Dynamic Typing

Goal: See how typed vs untyped variable declarations work

Language:

- Python (dynamic typing)
- Type Script (static typing)

Exercise 2A: Dynamic Typing (Python)

Fill the blanks!

```
# Python allows changing types freely
a = 10   # a is now an integer
a = "hello"  # a is a string
print(a)
```

Questions:

1. What type did a start as? → Integer
2. What types did a become later? → string
3. Does Python allow the variable type of to change?
   ans:- Yes

Exercise 2B: Static Typing (Type Script)

Fill in the blanks:

```
let count : number = 5;
count = "hello";  // will this work? → no
```

Questions:-

1. What type is count restricted to? → Integer
2. Why does count = "hello" cause an error?
   ans:- Type 'string' is not assignable to type 'number'.

3. Type inference

Goal: See how a language guesses types for you.

Language: Type Script or Haskell-style pseudocode.

Exercise 3: Type inference (Type script)

Fill in the blank:

```
let message = "Welcome!";
let price = 500;
console.log(typeof message); // what type did it infer? → string
console.log(typeof price); // what type did is infer? → Intiger
```

Questions:

1. what type did the compiler infer for message? → string
2. what type did it infer for price? → Intiger
3. Why is type inference helpful for beginners?
   → It reduces typing, avoids redundancy, and still keeps types safe.

4. Memory Management (Stack, Heap, Garbage Collection)

Goal: Understand where value live in memory and how unused memory is cleaned.

Language: Python (because GC is automatic)

Exercise 4: Stack vs Heap (conceptual + code)

Fill in the blanks.

```
def add(a, b):
    ** a and b live on the stack
    result = a + b
    return result ** result also lives on the hip temporarily
** A list lives on the stack because it can grow number
numbers = [1, 2, 3]
** The variable 'numbers' itself is stored on the hip
```

Garbage Collection Example.

text = "Hello"

text = "Goodbye" ✻ "Hello" become unreachable

Questions:-

1. Stack is used for:

→ local variable

2. Heap is used for

→ big flexible object

3. What happens to "Hello" after rebinding?

ans:- The reference is changed to point to a new value, and "hello" becomes unused and may be removed by garbage collection if no other reference point to it.

2.2 Composite Data types

- Arrays, records, unions, and algebric data types
- heneric programming (templates, monomorphization)

language used:

- Python for arrays list and records
- TypeScript for unions & generics (because it simple and readable)

1. Arrays, Records, Unions, and ADTs

Exercise 1 - Arrays (Python)

✻ Create a list of three colors

colors = ["red", "blue", "green"]

✻ Add a new color

colors.apend ("green")

✻ print the updated list

print (colors)

Questions:
1. What color did you add to the list? → green
2. How many elements does the list have now? → 3

Exercise 2 - Records (Python dictionaries)
* Create a simple record describing a student

```
student = {
    "name": "Alice",
    "age": 21,
    "grade": "A"
}
```

* Update the student's grade

```
student["grade"] = "D"
print(student)
```

Questions:
1. What age did you give the student? → 21
2. What is the ~~new~~ student's new grade? → "D"

Exercise 3 - Union Types (Type Script)

```
// A value can be either a number OR a string
let value: number | string;
value = 10;
value = "hello";
value = hello2; // try assigning a number OR a string
```

Questions:
1. What did you assign to value in the blank? → hello2
2. Would assigning true work? Why or why not?
ans:- No, because true is a boolean, and the union type number | string allows only numbers or strings, not booleans.

Exercise 4 - Algebraic Data Type (Basic Tagged Union)

```
// A shape can be a circle or a Rectangle
type Shape =
  | { kind: "circle", radius: number }
  | { kind: "rectangle", width: number, height: number };
// Create one circle
let s1: Shape = { kind: "circle", radius: 7 };
// Create one rectangle
let s2: Shape = { kind: "rectangle", width: 5, height: 4 };
```

Questions:-

1. What value did you put for the circle's radius? → 7
2. What value did you put for the rectangle's height? → 4.
3. Why is this called an "algebraic data type"?
   ans:- Because it combines data using product or sum.

Exercise 5 - product Type

Goal:
Understand that a product type combines multiple pieces of data together (like a record or struct).
A product type groups field together, like:

```
type Point = {
    x : number,
    y : number
};
```

Now complete this small exercises:

```
// A product type representing a book
type Book = {
    title : string,
    page : number
};
```

```
// create one book
let myBook: Book = {
  title: Hero,
  page: 192
};
console.log("Book title:", myBook.title);
console.log("Number of pages:", myBook.pages);
```

Questions:

1. What title did you give the book? → hero
2. How many pages did you enter.? → 192
3. Why is this called a "product type"?
   ans. Because it combines several fields together into one types.
2. Generic Programming (Templates, Monomorphization)

Exercise - Simple Generic Function

```
// A generic function that returns the first element of an
array function firstElement<T>(arr: T[]): T {
    return arr[0];
}
// use with numbers
let nums = [1, 2, 3];
let firstNum = firstElement(nums); // T becomes number
// use with strings
let names = ["Alice", "Bob"];
let firstName = firstElement(names); // T become string
console.log(firstNum);
console.log(firstName);
```

Questions!

1. When calling first Element (nums), what types does T become?
   → number
2. When calling first Element (names), what types does T becomes?
   → string.
3. What does "monomorphization" mean?
   ans!- It means the compiler creates a concreate version of the generic function for each used type.

3. Control structures
   - Structured vs. unstructured control flow
   - Exception handling mechanisms
   - Coroutines and continuations

1. Structured vs. Unstructured control flow

Goal:
Understand the different between structured flow (if, while, for) and unstructured (jump-like flow (eg. break, continue, early return).

Excercise - Structured vs Unstructured flow.

** structured: Using a loop to print number 1 to 5

```
for i in rong (1, 6):
    print (i)
```

** Unstructured: Jumping out early using break

```
for j in rongee (1, 10):
    if j == 7 : ** stop when j reaches this number
        break
    print ("j =", j)
```

Questions:
1. what number did you choose to break the loop? 7
2. which part is structured?
ans!- The normal for loop iteration
3. which part is unstructured?
ans!- The jump using break
4. why is "structured" flow easier to read?
ans!- Berause it flows a predictable top to bottom control path without sudden jumps.

2. Exception Handling Machanisms
Goals:
Understand try/except and what happens when an error occurs.
language: Python
Exercise- Catching Errors
** Task: Fix the crash using try/except

```
number = [10, 20, 30]
try:
    index = 2 * choose an index (0-3)
    print (numbers [index])
except indexerror:
    print ("Oops! That index does not exist.").
```

Questions:
1. what index did you choose? ⇒ 2
2. If you picked an out of range index, what printed?
→ "Opps! That index does not exist."
3. Why is exception handling useful?
To prevent programs from crashing when something unexpected happens.

5. Coroutines and Continuations

Goals:

lets student see how a continue pauses and resumes: understand "what happen next" (continuation).

Exercise - Simple Coroutine Greeter

```
## simple greeter coroutine
def greeter():
    while True:
        name = yield    ## loop forever, so coroutine doesn't end
        print("Hello,", name)  ## pause and wait for a name
## create and prime the coroutine
g = greeter()
next(g)                 ## prime it, get it ready to receive a value
## send some names
g.send("Alice")
g.send("Bob")
g.send("Charlie")
g.send("Prem")          ## send your name
## program continues normally
print("program continues here!")
```

Questions:

1. What name did you send? → "Prem"
2. When yield is hit, does the function stop forever or pause?
   ans:- Pause
3. After printing the greeting, where does the program continue?

ans:- At the next line after g.send(...), which is print("program continues here.!")

Continuation

Goal:

Understand the idea of "what happens next" in a program. a continuation is just the rest of the program after a pause

Exercise - Simple Continuation with a Generator

```
import time
def simple_continuation():
    print("step 1: start")
    time.sleep(2)  ** pause for 2 seconds
    yield          ** save the rest of the program
    print("step 2: Resume after pause")
    time.sleep(2)  ** pause for 2 seconds.
    yield
    print("step 3: End")
** Create the continuation
c = simple_continuation()
** step 1.
next(c) ** Pause after printing step 1
** step 2
next(c) ** pause after printing step 2
** step 3.
next(c) ** prints step 3
print("program continues here!")
```

Questions to fill in

1. What print when you call next(c) the first time? → start
2. What prints when you call next(c) the second time? → Resume after pause
3. What prints when you call next(c) the third time? → End.
4. What is printed after the continuation ends? → program continue here!
5. In one sentence! what does a continuation represent?
   A continuation is the "rest of the program" saved so it can run later.

How it works

1. Each yield pause the function and remembers what to do next.
2. Calling next() resumes from the last yield.
3. When the generator ends. Python raise StopIteration (but the program can continue).